Cidades Inteligentes

Sistema de Monitoramento Escolar - Documento de Ideias para a Equipe

Projeto de Extensão Universitária - Versão 1.0 (rascunho para discussão da equipe)

1. Sobre este documento

Este documento organiza as ideias discutidas até agora sobre um projeto de extensão na área de cidades inteligentes, em parceria com uma escola. O objetivo é dar ao time de 6 pessoas uma visão coesa do conceito, das possibilidades, do escopo sugerido para 1 semestre e da divisão de papéis - para que a decisão final seja coletiva e informada.

Nada aqui está fechado. O documento é um ponto de partida: a escolha do sistema de monitoramento concreto (ex.: lixo, como no exemplo principal) deve ser validada com a escola parceira antes de detalharmos o plano de execução.

2. Contexto e motivação

O projeto: extensão universitária que une a faculdade a uma escola, aplicando o conceito de cidades inteligentes (smart cities) em escala real e tangível.

A ideia central: uma escola é uma minicidade. Ela gera resíduos, consome energia e água, tem fluxo de pessoas, ruído e espaços que precisam de manutenção. Monitorar e otimizar esses recursos em escala escolar é exatamente o que cidades inteligentes fazem em escala urbana - com a vantagem de ser um ambiente controlado, seguro e com impacto social direto.

Exemplo concreto (caso principal): monitoramento de lixo. Sensores medem o nível de enchimento das lixeiras em tempo real, o sistema prevê quando cada uma vai encher, sugere rotas de coleta e engaja os alunos por meio de ranking de reciclagem por sala.

3. Visão geral do sistema: 4 pilares

O sistema se apoia em 4 pilares independentes - essa separação também define como a equipe se organiza:

Pilar	O que faz	Tecnologias típicas
1. Sensoriamento (IoT)	Coleta dados reais do ambiente: nível de lixo, presença, imagens	ESP32, sensor ultrassônico (HC-SR04), câmera, WiFi
2. Dados e API	Recebe, armazena e expõe os dados de forma estruturada	NestJS, Prisma, PostgreSQL
3. Inteligência (IA/ML)	Prevê, classifica e alerta sobre os dados coletados	Python, scikit-learn, XGBoost/Prophet, YOLO
4. Engajamento (extensão)	Leva o resultado para a comunidade escolar e gera impacto social	Dashboard web, ranking, relatórios, painel público

Fluxo dos dados: sensor -> API/banco -> dashboard + IA -> ações (alertas de coleta, ranking de salas, relatórios). Cada pilar conversa com o vizinho por interfaces bem definidas.

4. As ideias em camadas

As ideias foram organizadas em 4 camadas, da base até o diferencial. Cada camada agrega valor à anterior - e nenhuma depende de hardware caro para começar.

Camada 0 - A fundação (sem IA, mas necessária)

- **Lixeira inteligente:** ESP32 + sensor ultrassônico (custo aproximado R$45 por ponto) mede o nível de enchimento em % e envia via WiFi (MQTT ou HTTP) a cada poucos minutos.

- **Backend:** API (NestJS + Prisma + PostgreSQL) recebe, valida e armazena as leituras.

- **Dashboard:** mapa da escola com o nível de cada lixeira em tempo real + alertas quando passar de 80%.

Sem esta camada, não há dados - e sem dados, não há IA. É também a camada onde o modo simulação (roda tudo sem hardware) é construído.

Camada 1 - IA preditiva (maior impacto pelo menor custo)

- **Previsão de enchimento:** com o histórico de leituras, um modelo de séries temporais (XGBoost ou Prophet) aprende o padrão da escola e responde: "a lixeira do refeitório vai encher quinta às 14h". A coleta muda de reativa para preditiva.

- **Sugestão de rota de coleta:** com as previsões do dia, o sistema sugere a ordem ideal de visita às lixeiras (problema clássico de otimização, muito citado em entrevistas).

- **Detecção de anomalias:** lixeira que enche rápido demais, sensor que parou de responder, pico anormal de temperatura (risco de fogo). O sistema classifica e avisa.

Camada 2 - Visão computacional (o diferencial visual)

- **Classificação de resíduos por foto:** modelo YOLO ou CNN treinado para identificar plástico, papel, metal e orgânico a partir de uma foto do lixo.

- **Detecção de lixo fora da lixeira:** câmera apontada para o ponto de coleta detecta transbordo no chão e gera alerta - problema real da escola, fácil de demonstrar.

- Para reduzir custos, dá para treinar com fotos tiradas no próprio celular (transfer learning com dataset pequeno).

Camada 3 - Engajamento e impacto social (o coração da extensão)

- **Gamificação entre salas:** cada lixeira pertence a uma sala/área; ranking de quem mais recicla. Garante uso real e impacto social - o que a faculdade avalia em extensão.

- **Relatório mensal automático:** resumo gerado por IA para a direção ("a escola gerou X kg de reciclável; a sala Y melhorou 30%").

- **Chatbot educativo:** via Telegram/WhatsApp: "qual a lixeira mais próxima com espaço livre?" + dicas de reciclagem.

- **Totem na escola:** TV com o dashboard em tempo real - visual forte para fotos e relatórios.

Camada	Exemplos	IA?	Complexidade
0 - Fundação	Lixeira inteligente, backend, dashboard	Não	Baixa (base de tudo)
1 - IA preditiva	Previsão de enchimento, rotas, anomalias	Sim (séries temporais)	Média
2 - Visão computacional	Classificação de resíduos, transbordo	Sim (YOLO/CNN)	Alta
3 - Engajamento	Gamificação, chatbot, relatórios, totem	Parcial (IA generativa)	Média

5. Escopo recomendado para 1 semestre

Em 1 semestre, a decisão mais importante é o que fica FORA. Um semestre com hardware real + IA + extensão + GitHub só funciona com escopo enxuto (princípio YAGNI: nada que não seja necessário agora).

INCLUIR (essencial)	ADIAR (stretch goals)
1 lixeira física real (ESP32 + sensor) para provar o hardware	Visão computacional (YOLO) - documentar como 'futuro' no README
Modo simulação no Docker (todo o resto roda sem hardware)	Chatbot e totem na escola
Backend NestJS + Prisma + PostgreSQL	Relatório automático por IA
Dashboard simples (mapa + nível + alertas)	Mais de 2 lixeiras físicas
IA: previsão de enchimento (o aumento de escopo)	
Gamificação leve (ranking por sala) - justifica a extensão	
CI/CD + README + demo no GitHub	

6. Cronograma sugerido (16 semanas)

Fase	Semanas	Entrega principal
1. Fundação	1-4	Backend + modo simulação + dashboard; todo o fluxo funciona sem hardware
2. Hardware real	5-8	ESP32 + sensor ultrassônico enviando dados ao backend; lixeira real no dashboard
3. Inteligência	9-13	Previsão de enchimento em 3 degraus didáticos: (a) baseline ingênuo, (b) regressão com features de tempo no scikit-learn, (c) XGBoost ou Prophet; alertas de coleta preditiva
4. Extensão e GitHub	14-16	Demo na escola com a lixeira real, ranking das salas, relatório de impacto e polimento do repositório (README com fotos e diagrama, vídeo demo, CI verde)

Nota sobre ML: para 1 semestre, XGBoost/Prophet entregam mais que redes neurais (LSTM) com uma fração do esforço. Cada degrau vira um notebook Jupyter publicado no GitHub - portfólio independente.

7. Equipe de 6 pessoas: como o time muda o projeto

O calendário da faculdade não muda com o tamanho do time: 1 semestre continua sendo 1 semestre. O que 6 pessoas compram é ESCOPO e QUALIDADE, não prazo. Atenção à lei de Brooks: adicionar pessoas atrasa projetos atrasados, porque coordenação e integração consomem tempo.

A divisão natural, com fronteiras claras entre módulos (ninguém pisa no trabalho do outro):

Papel	Responsabilidades	Pessoas
Hardware / Embedded	ESP32, sensores, firmware, instalação na escola	1-2
Backend / API	NestJS, banco, integração com sensores e simulação	1-2
Dashboard / UI	Mapa, ranking, alertas	1
IA / ML	Previsão de enchimento + notebooks	1
Integrador	GitHub (PRs, CI, deploy), README, relatório de extensão	1

Ganho real do time de 6: a Camada 2 (visão computacional) volta a ser viável no mesmo semestre, 3-4 lixeiras físicas em pontos diferentes da escola viram opção (mais impacto real), e a entrega sai com testes, CI e documentação de nível profissional.

8. GitHub e currículo: os diferenciais

O que separa este projeto de 90% dos projetos de faculdade no GitHub:

- **Modo simulação:** Docker Compose que roda TUDO sem hardware (dados sintéticos gerados por script). Um recrutador roda o projeto em 5 minutos sem comprar nada.

- **Dataset anonimizado publicado** (GitHub ou HuggingFace) - mostra cuidado com dados.

- **Monorepo com módulos separados + git desde o dia 1** (branches, pull requests, code review). Coordenar 6 pessoas com Git num projeto real é, por si só, uma linha forte no currículo.

- **CI/CD com GitHub Actions, testes e README** com diagrama de arquitetura e fotos reais.

- **Notebooks de ML** como portfólio independente ("aprendi séries temporais na prática").

- **Frase-chave no currículo:** "deploy de ML ponta a ponta: IoT -> API -> dashboard" - raríssimo entre estagiários.

- **Relatório de extensão** documentando impacto social real (escola + gamificação), que é o que a faculdade avalia.

9. Riscos e pontos de atenção

Risco	Mitigação
Muita gente mexendo nos mesmos arquivos	Monorepo com módulos separados + code review obrigatório
Escopo estourando no meio do semestre	Regra do time: feature nova só entra se uma for cortada (YAGNI)

Risco	Mitigação
Dependência de hardware (atraso, peça queimada)	Modo simulação sempre funcional; hardware é complemento, não dependência
Poucos dados para o modelo de ML	Começar com dados sintéticos; o modelo real melhora a cada semana de uso
Integração deixada para o final	Integrar desde a semana 1 (fluxo simulado já ponta a ponta)
Equipe desalinhada sobre o objetivo	Este documento + reunião de kickoff para fechar papel de cada um

10. Próximos passos

1. Fechar com a escola parceira qual sistema de monitoramento concreto (ex.: lixo) será o foco.

2. Validar o prazo oficial do projeto com a faculdade (início e fim do semestre letivo).

3. Definir com a equipe os papéis da seção 7 e o nome do projeto/repositório.

4. A partir daqui, desenhar o plano detalhado de execução: arquitetura, tarefas semana a semana e stack final - incluindo o passo a passo didático para a parte de IA.